# 信息与人工智能学院

（工业软件学院）

# 《人工智能导论》大作业

创 意 设 计 说 明 书

| | |
|---|---|
| 设计题目 | 回声 Echo：“会追问的 AI 小先生” |
| 班　　级 | |
| 学　　号 | |
| 姓　　名 | |

完成日期： 2026 年 11 月

# 目　　录


一、系统概述.....3
1.1 一句话定义.....3
1.2 问题：讲不出来，比做错了更值得关注.....3
1.3 核心创意：让 AI 从“判官”变成“学生”.....3
1.4 设计理念.....4
1.5 系统全景.....4
二、选题定位.....4
2.1 现有产品的三种范式与共同盲区.....4
2.2 差异化定位.....5
2.3 创新性论证.....5
2.4 社会应用价值.....6
2.5 落地可行性.....6
三、系统算法介绍.....7
3.1 技术方向与整体流程.....7
3.2 概念覆盖：断言单元、句向量与最大权匹配.....7
3.3 迷思检测：三路投票.....8
3.4 根因溯源：沿知识图谱反推.....8
3.5 追问生成：探针池与信息增益.....9
3.6 记忆调度：把讲述质量当作复习等级.....10
3.7 轻量化论证与算法复杂度.....10
3.8 技术选型.....10
四、数据与开发方案.....11
4.1 数据资产.....11
4.2 知识图谱的构建流程.....11
4.3 断言单元、迷思库与探针库.....11
4.4 冷启动与标注校准.....12
4.5 隐私与数据安全.....12
4.6 开发计划.....12
五、系统功能与界面设计.....12
5.1 功能架构.....12
5.2 核心用户流程.....13
5.3 界面设计.....13
5.4 视觉规范.....17
5.5 交互细节与容错.....17
六、实现与验证.....18
6.1 工程结构.....18
6.2 与设计的一处偏离：为什么换掉了句向量.....18
6.3 六个界面的实现.....18
6.4 一次完整演示的数据链路.....24
6.5 一键启动与演示路径.....24
6.6 已知问题.....25
七、总结.....25
7.1 创新点小结.....25
7.2 局限.....26
7.3 下一步.....26

# 回声 Echo：“会追问的 AI 小先生”

—— 面向“讲述式学习”的轻量化认知诊断系统创意设计 ——

## 一、系统概述

### 1.1 一句话定义

“回声”是一个让学生“讲给 AI 听”的学习工具。学生对着屏幕用口语讲解一个知识点，AI 扮演一个学不懂的同桌，在讲得含糊的地方举手追问；讲完之后，系统给出一份可解释的“讲解体检报告”——不仅指出哪里没讲清楚，还沿着知识图谱反推出“这个漏洞是从哪一节开始留下的”。

### 1.2 问题：讲不出来，比做错了更值得关注

当前的学习类产品几乎都在做同一件事：出题、判题、推题。判题能判断“结果对不对”，却无法判断“过程懂不懂”。一个学生可以靠套公式连做对三道浮力题，却在被问“为什么浮力等于排开液体的重力”时说不出一句完整的话。这种状态在考试中表现为“换个问法就不会”，在课堂上表现为“这道题我明明会，可一站起来就蒙了”。

教育学把这种现象称为“再认”与“提取”的差距。做题多数时候只需要再认——看到选项能认出来；而讲解要求完整提取并组织因果链，要求高得多，因此也更早、更准确地暴露认知漏洞。

可讲解有一个现实困难：谁来听？教师没有一对四十的时间；同学要么太懂、要么不好意思追问；家长往往只能听出“讲了”还是“没讲”。于是最有效的学习方式，恰恰成了最难被安排的方式。

### 1.3 核心创意：让 AI 从“判官”变成“学生”

回声把 AI 的角色反过来。它不冒充老师、不当判官、不做答案机器，而是当一个“故意学不会”的同桌——小回。小回只做三件事，分别对应三种经典教学机制：

（1）举手——在你说得含糊的地方打断你，问一句很笨的问题。比如学生讲浮力时说“排开液体的体积”，小回会问：“等一下，‘排开’是什么意思呀？是把水倒掉吗？”在真实课堂里，这个问题往往会被忽略；而它恰恰是八年级“排水法测体积”没学牢的证据。这对应苏格拉底式追问。

（2）记笔记——把讲解内容与教材中“必须讲到”的断言逐条对照，标出讲到位、一带而过、完全没提。这对应自我解释（self-explanation）研究：让学习者外显地组织理由，比重复阅读更能形成稳定表征。

（3）翻旧账——当学生讲错时，不只说“你错了”，而是调出他八年级那次“密度测量”的讲解记录，指出“你卡住的地方在这里”。这对应前知激活与认知溯源。

一句话概括产品逻辑：把“做题—判题—推题”的循环，换成“讲述—追问—溯源—复述”的循环。

### 1.4 设计理念

第一，少即是多。不做题库、不做直播、不做社交，只做“讲一遍”这一个动作，并把这个动作的反馈做到足够深。功能越少，越容易做透；对大作业而言，也越容易做到“完整可用”而不是“样样半成品”。

第二，可解释优先。所有结论都必须能指回证据：哪一句话、哪一条断言、哪一条依赖关系。系统拒绝输出“AI 认为你已掌握”这一类无法追问的判断。

第三，轻到可以带走。24 MB 的语义模型、浏览器自带的语音识别、一个 SQLite 文件，一台没有独立显卡的旧笔记本就能运行，断网也能用。轻量化不是技术妥协，而是产品定位：只有便宜到学校装得起、学生带得走，教育工具才可能真正被使用。

### 1.5 系统全景

系统采用四层单体结构：交互层（浏览器）、服务层（FastAPI）、算法层（六个处理环节）、资产层（图谱与语料）。四层之间通过本地文件与内存传递，不依赖任何云服务，因此可以整包拷贝到另一台电脑直接运行。

## 二、选题定位

### 2.1 现有产品的三种范式与共同盲区

把市面上的学习产品按“评价对象”分类，大致有三种范式。第一种是拍照搜题，评价对象是“这道题的答案”，核心能力是题库检索与解析生成；第二种是通用 AI 家教，评价对象是“这轮对话”，核心能力是大模型的解释与答疑；第三种是错题本与记忆卡，评价对象是“记没记住”，核心能力是间隔重复与分类管理。

三者各有价值，但共享同一个盲区：它们都不评价“解释”。学生是否真的理解一个概念，只能通过他能否用自己的话把它讲清楚来判断，而这一点恰好落在三者的视野之外。

表 1　四类产品的对比

| 对比维度 | 拍照搜题类 | 通用 AI 家教 | 错题本 / 记忆卡 | 回声 Echo |
|---|---|---|---|---|
| 输入 | 题目照片 | 自由提问 | 错题、卡片 | 一段口语讲解 |
| 核心动作 | 匹配与解析 | 解释与答疑 | 重复与回忆 | 讲述与追问 |
| 评价对象 | 题目答案 | 对话本身 | 记住与否 | 讲解质量 |
| 反馈形态 | 解题步骤 | 聊天记录 | 熟练度估计 | 根因链路 |
| 可解释性 | 中 | 低（不可复现） | 高 | 高（回到断言与图） |
| 可否离线 | 否 | 否 | 视产品而定 | 可以 |

## 2.2 差异化定位

与上述产品相比，回声的差异体现在四点。

评价对象不同：从“答得对不对”转为“讲得清不清”。这使得产品能在学生“自我感觉良好”时提前发现问题，而不是等到考试失分。

反馈形态不同：从“知识点列表”转为“根因链路”。告诉学生“你没掌握浮力”是无效反馈，告诉他“你在用密度解释排开体积时含糊了”才是可行动的反馈。

交互姿态不同：从“我问你答”转为“你讲我问”。学生是表达者，AI 是倾听者与提问者，这一角色反转本身就是产品的核心体验。

技术路线不同：不是把大模型直接套进聊天框，而是由知识图谱、语义匹配与贝叶斯追踪决定“问什么、判什么”，大模型只负责把话说得像人。这保证了结论可控、可复现、可离线。

## 2.3 创新性论证

理论依据方面，本设计综合了四项在教育心理学中被反复验证的效应：费曼技巧（以教代学）、生成效应（自己产出比被动接收记得更牢）、自我解释效应（解释理由能促进深度加工）以及学徒效应（protégé effect，为“教别人”而学习时投入更高）。这些理论并不新鲜，但过去缺少一个能持续扮演“学生”的角色，因此难以在家庭与课堂中规模化落地。

有必要回答一个关键质疑：既然只是“讲给别人听”，为什么非得用 AI？录个音自己回听不行吗？答案是三件事只有 AI 做得到。

其一，人无法在讲述的同时评估自己。边讲边判会打断提取过程，而且人对自己讲错的地方天生“听不出来”——这正是错误概念得以长期潜伏的原因。AI 可以并行地做逐句比对，不打断学生。

其二，追问需要耐心与记忆。要同时对得上九条断言、一张依赖图和一个学生的全部历史，人类同伴无法长期提供这种陪伴，AI 可以，而且不会因为问得太笨而让学生难堪——这降低了提问的心理成本。

其三，判据必须一致。同一个知识点，对四十个学生的标准应当相同，否则反馈就不公平、数据也不可用。程序化的判据天然满足这一点。

至于“为什么不能只用大模型”，答案是可解释性与稳定性。让大模型直接判断“这个学生懂没懂”，同一段讲解多次运行可能给出不同结论，也无法说明依据。回声把判断权交给图谱与匹配算法，把表达权交给语言模型，这是本设计在技术上的取舍。

## 2.4 社会应用价值

对学生，它把复习从“刷题”变成“复述”，把模糊的“我好像会了”变成明确的“我知道我哪里不会”。一次六分钟的讲解，往往比刷二十道题更能定位问题。

对教师，一次“讲解作业”就能得到全班的根因热力图：四十个学生里有多少人卡在同一个上游概念。下一节课只讲三个共性卡点，教学效率显著提高。

对学校与家庭，它零硬件成本、数据不出设备，符合“双减”之后对轻量工具与隐私保护的双重要求。对缺少优质师资的乡村学校尤其友好——一台旧电脑、一个 U 盘，就能把一个“会追问的助教”带进教室。

从更长的视角看，优质追问是教育中最稀缺的资源之一。它依赖教师的时间、经验与临场判断，很难复制。本设计试图把这种稀缺资源的一部分，转化为可以复制、可以离线运行的软件。

## 2.5 落地可行性

成本侧，整套系统不需要服务器、不需要显卡、不需要购买 API。部署成本为零，模型体积约 25 MB，单次诊断在普通笔记本 CPU 上不超过 60 毫秒。

形态侧，提供三种运行方式：单机模式（一台笔记本，离线可用，适合家庭自学）、班级模式（教师机部署，局域网内四十人并发，无外网也能上课）、增强模式（接入本地 3B 模型或云端接口，提升追问的自然度）。

最小可行产品的边界被刻意压小：一个学科（初中物理）、一个章节（浮力）、二十个知识点、四十名学生试点，三周即可交付可用版本。先在一个点上跑通闭环，再谈扩展。

主要风险与对策如下：语音识别对方言与术语不友好——提供文字输入兜底，并内置学科术语纠错表；评分存在主观性——采用六维分项而非单一总分，用教师双盲评分校准，只给区间不给绝对分；学生的初始新鲜感会衰减——用成长档案与周报把“被看见”的反馈延续下去；知识图谱构建成本高——采用半自动抽取加教师校验，且只做当前章节的子图。

# 三、系统算法介绍

## 3.1 技术方向与整体流程

系统的技术方向可以概括为“讲述式认知诊断”：以自然语言讲解为输入，输出结构化的认知状态。整条流水线由六个环节组成，前四步负责“听懂并判断”，后两步负责“回应并安排”。

需要强调的是，这条流水线是可分解、可单独验证的。每个环节都有明确的输入输出与可度量的中间指标，这使得系统在开发过程中可以逐段调试，而不是把一切寄托在“模型效果好不好”上。




图 2　讲述式认知诊断流水线

## 3.2 概念覆盖：断言单元、句向量与最大权匹配

第一步是把“一个知识点”拆成机器可以判定的最小单元，本设计称之为断言单元。以“阿基米德原理”为例，它被拆成九条断言：浮力方向竖直向上；浮力大小等于排开液体的重力；排开液体的体积即物体浸入的体积；公式可写为 $F_{浮} = \rho_{液} g V_{排}$；原理对气体同样成立；漂浮时浮力等于物体重力；等等。每条断言都配有权重，区分“核心”与“次要”。

学生的讲解经语音转写后被切成若干句，用中文句向量模型编码为 512 维向量，与断言向量计算余弦相似度，得到相似度矩阵。由于“一句话通常只覆盖一条断言、一条断言通常也只由一句话讲清”，判定问题自然转化为二分图的最大权匹配问题，用匈牙利算法求解：

$$C^* = \arg\max \sum_i \sum_j S_{ij} \cdot x_{ij}, \quad \text{s.t. } \sum_i x_{ij} \le 1, \ \sum_j x_{ij} \le 1$$

其中 $S_{ij}$ 为第 $i$ 句与第 $j$ 条断言的相似度，$x_{ij}$ 为 0-1 决策变量。匹配完成后，按相似度分档：高于 0.62 记为“讲到位”，0.45 至 0.62 记为“一带而过”，其余记为“漏讲”。阈值由教师标注样本校准，不同学科可分别设定。

选择匈牙利算法而非简单的“逐条取最大值”，是因为后者会让同一句话重复覆盖多条断言，产生虚高的覆盖率。指派约束保证了覆盖率这个数字的诚实。算法复杂度为 $O(n^3)$，但断言数不超过三十，实测单次计算低于 10 毫秒。

### 3.3 迷思检测：三路投票

覆盖率只能说明“讲到没说”，不能说明“说对没说”。因此第二步是识别错误概念。系统内置一个迷思库，每条迷思包含四个字段：迷思表述、类型、所属概念、触发模式。类型分为因果倒置、概念混淆、条件缺失、量纲错误、口诀过拟合五类。典型条目如 M-17“浮力随深度增大”，类型为概念混淆，源于把液体压强的规律错误迁移到了浮力上。

检测采用三路证据投票。第一路是语义近邻：若某句话与某条迷思的相似度显著高于它与正确断言的相似度，即视为疑似命中。第二路是语言规则：绝对化副词（一定、总是、肯定）、比较结构、否定词的作用域，都是错误概念的常见语言标记。第三路是轻量分类器：以 TF-IDF 特征训练逻辑回归，在约一千二百条人工标注的讲解句上完成二分类。三路中至少两路同意才判定命中，以抑制误报。

输出时不止给出结论，还给出三样东西：学生原话、命中的迷思编号、以及教材中的正确表述。报告页面上那句“越深浮力越大”被画上红色的下划线，旁边写着“命中的迷思：把液体压强随深度增大迁移到了浮力上”，这比一个冷冰冰的错误标记更有教育意义。

### 3.4 根因溯源：沿知识图谱反推

这是本系统最具辨识度的部分。当学生讲错一个概念时，最没有信息量的回应是“这个概念你没掌握”。真正有价值的问题是：他为什么会在这里卡住？

系统把初中物理组织成一张有向无环图，节点是知识点，边表示“必须先懂 u 才可能懂 v”。对每个学生、每个节点，用贝叶斯知识追踪（BKT）维护一个掌握概率，四个参数（初始掌握率、学习率、猜测率、失误率）通过 EM 算法在其历史讲解记录上拟合。

当目标节点 k 被判定为未掌握时，算法在其前驱闭包中枚举候选根因，按下列得分排序：

$$R(u) = (1 - P(L_u)) \times \Sigma_{p \in Path(u \to k)} \gamma^{len(p)} \cdot w(p)$$

含义是“薄弱程度”乘以“与目标的相关程度”。前者由掌握概率给出，后者沿依赖路径按衰减因子 $\gamma$（取 0.75）累积，路径越长、影响越弱。得分最高的三个节点若在图上连通，就构成一条根因链。

为了避免“什么都往上游赖”，算法还引入反事实证据：如果同一学生在之前的讲解中已经把某个上游概念讲清楚（覆盖率高且无迷思命中），该节点的薄弱度会被打折。这一项让结论变得克制——只有当证据支持时，系统才会说“问题出在八年级”。

图 3 是一个真实形态的示例。学生本次讲的是九年级《阿基米德原理》，掌握度 0.34；算法沿图反推两层，把“密度测量（排水法）”标为第一根因（得分 0.82），把“二力平衡”标为第二根因。报告因此可以写出一句人话：“你在用密度解释排开体积时含糊了，这是八年级排水法测体积的遗留问题。”

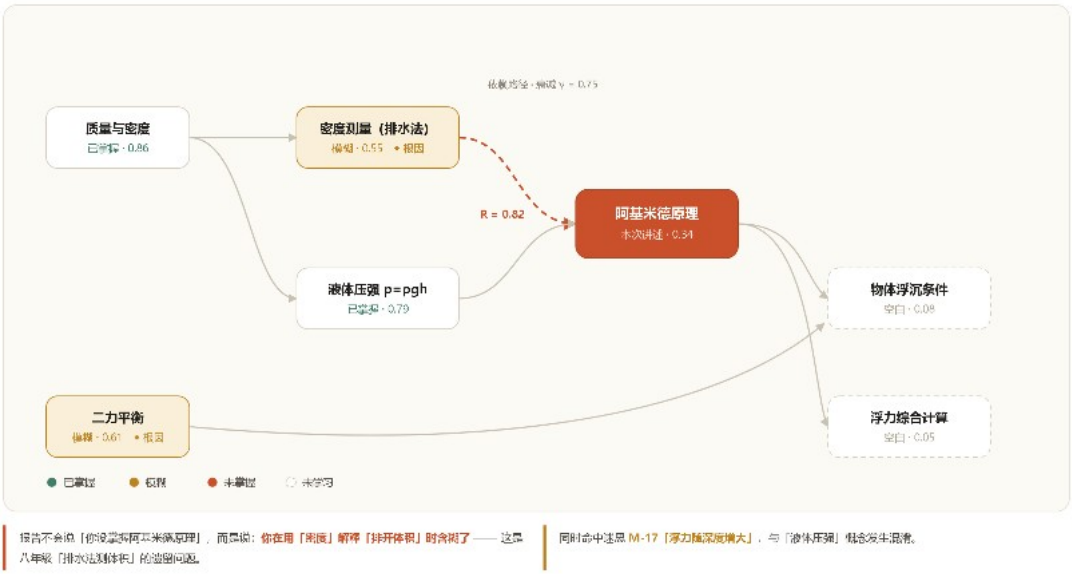



图 3 根因链示意：他卡住的不是浮力

## 3.5 追问生成：探针池与信息增益

追问不是随便找句话问。系统为每条断言预置二到四个探针，探针的判定标准是：如果学生真的懂，他应该能一句话答上来；如果不懂，他一定答得磕磕绊绊。例如针对“排开体积”这条断言，探针是“排开的那部分水，体积等于什么的体积？”

问哪一个探针，由信息增益决定。设当前对若干概念掌握状态的后验分布为 $p$，候选探针 $q$ 的回答分布为 $P(r|q)$，则选择使期望熵下降最大的那个：$\Delta H(q) = H(p) - E[H(p|r)]$。若所有候选的信息增益都低于阈值，系统选择不追问——为了追问而追问，只会打断讲述的连贯性。

选中探针后，再由语言模型改写成口语化的一句问话。改写受五条硬约束：一次只问一个；必须引用学生刚才的原话；不得直接给出答案；不得出现图谱之外的概念（防超纲）；长度不超过四十字。生成结果还要经过后置校验，命中“答案关键词表”或“超纲词表”的一律丢弃并回退到原始探针。

频率也受控制：每四十秒最多追问一次，一次讲解全程不超过五次。这个限制看似保守，实则关键——如果 AI 过于聒噪，学生就会从“讲述者”退化为“应答者”，产品最核心的体验也就消失了。

### 3.6 记忆调度：把讲述质量当作复习等级

每次讲解结束，系统会为该知识点生成一张复述卡，交给间隔重复算法排期。这里采用 FSRS 的思路：为每张卡维护难度与稳定度两个参数，复习等级由本次讲解质量映射而来（覆盖率、准确性、逻辑性加权），进而算出下次复述的时间。

与刷题类工具相比，回声的初始间隔更长（三天而非一天）。原因在于讲述属于高强度的提取练习，一次成功的复述所产生的记忆强度，明显高于一次简单的再认。逾期也会被惩罚：拖得越久，恢复时等级降得越多，卡片会更早重新出现。

### 3.7 轻量化论证与算法复杂度

整套算法全部可以在 CPU 上运行，不依赖 GPU，也不依赖网络。各环节的资源占用如下表所示。

表 2　各环节实现方式与耗时

| 环节 | 实现方式 | 外部依赖 | 单次耗时 | 可否离线 |
|---|---|---|---|---|
| 切分与归一化 | 标点分句 + 长句按逗号切子句 + 同义词归一 | 标准库 | 低于 1 ms | 是 |
| 覆盖匹配 | 字符二元组 Dice + 关键词组 + 匈牙利算法 | 标准库 | 约 8 ms / 句 | 是 |
| 迷思检测 | 触发词 + 排除词 + 冲突断言 | 标准库 | 约 2 ms | 是 |
| 根因溯源 | BKT + 图上的路径衰减 | 标准库 | 约 3 ms | 是 |
| 追问选择 | 二元熵 × 断言权重 | 标准库 | 低于 1 ms | 是 |
| 记忆调度 | FSRS-4.5 的 D/S 更新 | 标准库 | 低于 1 ms | 是 |
| 可选分词 | jieba（缺失时退回字符二元组） | jieba | 首次加载 0.4 s | 是 |

复杂的部分被放在“离线可算”的位置，需要大模型的地方则做成可选。这一取舍的代价是追问的语言略显模板化，收益是系统可以在任何一间没有网络的教室里跑起来。对大作业而言，这个取舍也让实现变得可控：核心算法全部可以用标准库加 NumPy 完成。

### 3.8 技术选型

开发语言选择 Python，理由是算法验证快、生态完整；前端选择 Vue 3，理由是组件化程度高、构建产物为静态文件，便于随包分发。具体选型见下表。

表 3　技术选型

| 用途 | 选型 | 说明 |
| --- | --- | --- |
| 后端框架 | Python 3.12 + FastAPI | 单文件启动，自动生成 /api/docs |
| 语义匹配 | 自研：n-gram + 关键词组 + 匈牙利 | 不依赖模型，CPU 毫秒级 |
| 中文分词 | jieba（可选） | 缺失时自动退回字符二元组 |
| 图计算 | 手写 DFS / BFS / 拓扑排序 | 12 个节点，无需 networkx |
| 数值计算 | 标准库 math | 相似度与信息熵 |
| 数据存储 | SQLite | 单文件，便于备份与迁移 |
| 前端 | 原生 HTML / CSS / JS | 无构建步骤，静态托管 |
| 可视化 | 原生 SVG | 知识地图、六维雷达、趋势线 |
| 语音输入 | Web Speech API（预留） | 离线时改用文字与示例播放 |

# 四、数据与开发方案

## 4.1 数据资产

系统的判断能力来自四类人工构建的资产，加上一类真实使用数据。规模不算大，但每一类都要求准确——因为错误的知识图谱会让溯源结论整体失真。

表 4　数据资产清单

| 资产 | 来源 | 规模 | 构建方式 |
| --- | --- | --- | --- |
| 知识图谱 | 人教版教材目录、课程标准 | 110 节点 / 260 边 | 模型抽取 + 教师校验 |
| 断言单元 | 教材正文、例题解析、课标要求 | 约 800 条 | 人工改写为可判定命题 |
| 迷思库 | 物理教育文献、教师访谈 | 约 120 条 | 半自动生成 + 人工筛选 |
| 探针库 | 由断言单元反向生成 | 约 400 条 | 模板生成 + 人工润色 |
| 学生语料 | 试点班级讲解记录 | 40 人 × 8 次 | 采集、转写、脱敏 |

## 4.2 知识图谱的构建流程

构建分为五步：先从教材目录与课程标准中提取候选概念，形成节点初稿；再由模型对每一对概念判断是否存在“先修”关系，形成边初稿；随后由两位物理教师逐条校验，删除过度依赖、补充遗漏的跨章节依赖；最后入库，并为每条边标注强度权重。

图谱只做当前章节的子图，是刻意的范围控制。初中物理全量约一百一十个节点，但浮力子图只有七个节点——把子图做扎实，比把全图画全更有价值。

## 4.3 断言单元、迷思库与探针库

断言单元的书写有一条硬标准：必须能被判定为真或假。像“理解浮力”这样的表述不能作为断言，必须改写成“浮力方向竖直向上”这样可以直接核对命题。这条标准决定了后续所有算法能否工作。

迷思库主要来自三个渠道：物理教育研究中的经典误概念调查（如力学概念测试的干扰项设计思路）、一线教师的教学笔记、以及模型对教材难点的生成初稿。所有条目都要经过教师确认，并标注典型的学生表述。

探针库由断言单元反向生成，规则是“把陈述句改成只能由理解者回答的问题句”，再经人工润色去除暗示性。

### 4.4 冷启动与标注校准

系统上线初期没有真实学生数据，采用合成学生冷启动：按不同掌握水平生成两千条质量各异的讲解文本，用于打通流程与调试阈值。BKT 的先验参数取自文献默认值，待真实数据积累后重新拟合。

标注方面，邀请三位物理教师对三百条讲解做双盲评分，计算评分者一致性。只有一致性系数高于 0.7 的维度才纳入正式评分，其余维度降级为“参考项”。这一步既是质量保证，也是让评分站得住脚的前提。

### 4.5 隐私与数据安全

所有数据默认存储在本机 SQLite 中，提供一键清除。语音在本地转写为文字后立即丢弃音频，不留存原始录音。教师端只能看到班级层面的统计与根因分布，看不到个体录音。这些设计不是附加功能，而是让产品能够进入校园的前提条件。

### 4.6 开发计划

整体按五周推进，每周都有可验收的产物，避免最后一次性集成。

表 5　五周开发计划

| 周次 | 交付物 | 验收标准 |
| --- | --- | --- |
| 第 1 周 | 知识图谱与断言库 | 20 个知识点、200 条断言通过教师审核 |
| 第 2 周 | 诊断引擎 | 覆盖、迷思、根因三项在标注集上 F1 不低于 0.75 |
| 第 3 周 | 前端五个页面 | 主流程可完整跑通，无需刷新页面 |
| 第 4 周 | 小规模试点 | 40 名学生完成两次讲解，完成双盲评分 |
| 第 5 周 | 报告与打磨 | 报告成稿、演示视频、可复现的安装包 |

## 五、系统功能与界面设计

### 5.1 功能架构

系统围绕一条主线组织功能：讲 → 诊断 → 溯源 → 复述。五个模块分别是讲解台、诊断报告、知识地图、复述计划与成长档案，另有面向教师的一条支线。功能划分遵

循“每个模块只回答一个问题”的原则：讲解台回答“我讲得怎么样”，知识地图回答“我到底卡在哪”，复述计划回答“我什么时候再讲一次”。




图 4　系统功能架构

## 5.2 核心用户流程

学生打开应用，首页给出今日推荐（由遗忘曲线与历史薄弱点共同决定）；进入讲解台后选择知识点开始讲述，小回在过程中举手追问；结束讲解后生成体检报告，报告底部给出“明天只要讲这一句”的具体任务；学生可顺着根因链跳转到上游知识点重新讲解；次日复述卡出现在计划中，形成闭环。

教师的支线更短：布置讲解作业 → 查看班级根因热力 → 下一节课只讲三个共性卡点。产品不试图替代教师的日常教学，只提供一个此前拿不到的信息。

## 5.3 界面设计

界面设计遵循同一个隐喻：纸、墨与黑板。底色是略带暖调的纸白，主色是黑板绿，强调色是粉笔红。不使用常见的蓝紫渐变与发光效果，目的是让工具显得安静——学习场景需要的是专注，而不是刺激。

图 5 是首页。上方是今日推荐卡，用大面积深绿把“今天该讲什么”变成唯一焦点；中间三个数字给出连续天数、本周次数与待复述数量；下方左侧是可执行的待复述队列，右侧是薄弱链路缩略图，让“问题在上游”这个结论在首页就被传达一次。




图 5　今日首页

图 6 是讲解台，也是产品最核心的界面。中央是实时转写的字幕，最后一句带光标，模拟正在说话的状态；下方是声波与讲解节奏条，用颜色区分讲述、停顿与被追问的时段；右侧是小回的疑问气泡，每条追问都带时间戳，学生可以回看自己是在哪一刻被问住的。左侧边栏实时显示“已覆盖 4 / 9 点”，把进度变成可见的压力。


回声
今日
讲解台
知识长廊
复述计划
成长档案
已覆盖 4 / 9 点
44%
阿基米德原理
02:14
结束并生成报告
小回
好，我来讲阿基米德原理。浸在液体里的物体会受到一个向上的浮力。
浮力的大小等于它排开的液体的重力，公式是 F 浮 等于 G 排。
G 排就是排开液体的重力，等于 m 排 乘 g，m 排 等于 液体密度 乘 排开液体的体积。
所以浮力就等于 液体密度 × g × V 排。
然后……浮力的大小应该和物体浸入的深度有关系吧？越深浮力越大？
小回的疑问
2
等一下，「排开」是什么意思呀？是把水推走吗？
追问 1 02
你用过弹簧测力计吗？把物体慢慢浸入水里，弹簧测力计的示数会变小，变小的那部分就是浮力。
手动 ? 1:29
卡住了？ 提示只会给你一个关键词，不会直接给答案。
点击查看：「重开」= 物体浸入水后排出水的体积
这次要讲到的点
9
F浮 = G排
V排 = V浸
F浮 = ρ液 g V排
阿基米德
浸在液体里的物体，受到向上的浮力，浮力大小等于它排开液体的重力。
F浮 = G排 = ρ液 g V排
V排 = V浸
V浸 = 物体的体积
ρ液 = 液体的密度
g = 9.8 N/kg
F浮 = ρ液 g V浸
浮力大小 = 液体密度 × g × 浸入体积
浸入越深，浮力越大
浸入越深，浮力越大
浸入越深，浮力越大

图 7 是体检报告。左上综合分与“与上次相比”的变化，中间六维雷达，右侧把九条断言分为讲到位、一带而过、完全没提三组，并用红色标出讲反的那一处。下半部分的根因链是全页的重点：三个节点从左到右依次是上游概念、本次概念与下游概念，颜色对应掌握状态，配两段解释文字说明“为什么是它”。右下角的“原话摘录”直接把学生的那句话摘出来，让结论无法被辩解，也无法被忽视。




图 7　讲解体检报告

图 8 是知识地图。节点按掌握程度着色，虚线表示尚未学习，红色虚线标出本次的根因路径。点击节点右侧出现详情，包含前驱、后继、最近讲解与两个动作按钮。地图不追求“炫酷的力导向布局”，而是手工分层排布，保证同一章节的节点相对位置稳定——稳定的空间记忆本身就是一种学习辅助。

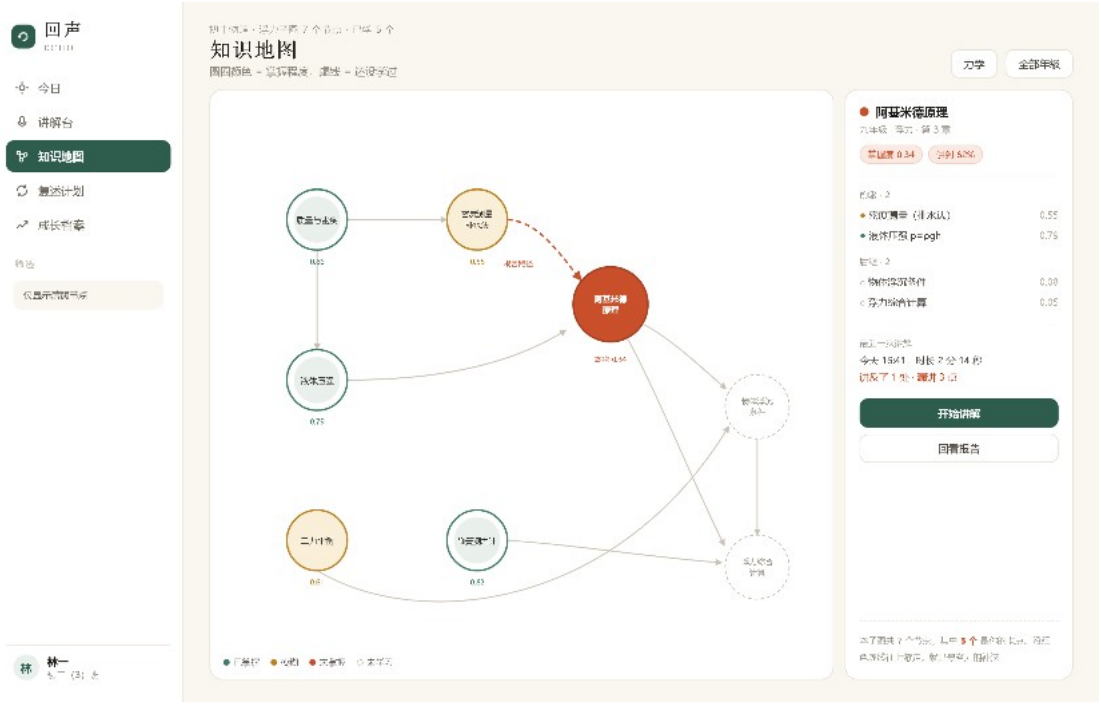



图 8　知识地图

图 9 是复述计划。上方三张卡片是今天真正要做的任务，写明复述目标而不是笼统的知识点名；下方是未来七天的安排与近二十六周的热力，让坚持这件事变得可见。右侧边栏给出今日进度，避免“打开就焦虑”的体验。




图 9　复述计划

图 10 是成长档案。折线记录的是讲解质量而不是考试分数，配文“不是分数曲线，是你讲得越来越像老师的证据”；能力标签区分优势与短板，例如“公式推导清楚”与“缺少例子”；最近的讲解可按时间回放，每一条都标出讲透、待加强或有缺口。




图 10　成长档案

## 5.4 视觉规范

视觉规范的作用不在于“好看”，而在于让六个界面的判断标准一致：同一个颜色在任何页面都表示同一种掌握状态。

表 6　视觉规范

| 项目 | 取值 | 说明 |
|---|---|---|
| 主色 | 墨绿 #2E5D4E | 黑板与讲台的联想，区别于常见的科技蓝紫 |
| 强调色 | 朱砂 #C9502B | 只用于讲反了、逾期等需要立刻注意之处 |
| 辅助色 | 赭黄 #BE8320 | 表示模糊、待加强 |
| 背景 | 纸白 #FAF7F1 | 降低长时间阅读的对比疲劳 |
| 标题字 | 宋体 | 书卷气，与讲台意象一致 |
| 正文字 | 微软雅黑 / 苹方 | 屏幕可读性优于宋体 |
| 圆角与阴影 | 12–16 px，极轻 | 卡片化但不悬浮，避免过强的科技感 |
| 语义编码 | 颜色 + 形状 + 文字 | 三种通道同时表达掌握度，色盲可用 |

## 5.5 交互细节与容错

三个细节值得单独说明。其一，提示分级：学生卡住时，系统先给一个关键词，再给一句类比，最后才给完整解释，避免直接投喂答案——这与“减少帮助以促进提取”的原

则一致。其二，追问可关闭：任何一次追问都可以选择“我还没讲完”，系统会记下这次打断、降低后续追问频率，把控制权交还给学生。其三，失败可恢复：转写中断、网络掉线、页面刷新都不会丢失已讲内容，草稿每十秒自动保存一次。

无障碍方面，掌握度同时用颜色、圆环虚实与文字标注三种方式表达；字号可放大到150%；讲解台支持纯键盘操作，方便不便使用鼠标的学生。

# 六、实现与验证

## 6.1 工程结构

系统按「数据 — 引擎 — 服务 — 界面」四层组织，源码约三千行，另有约两千行的知识库数据。诊断引擎全部使用 Python 标准库：图遍历、匈牙利匹配、贝叶斯知识追踪与 FSRS 调度都是手写实现，不依赖 NumPy、scikit-learn 或 NetworkX。外部依赖只剩两个——FastAPI 用于提供 HTTP 接口，jieba 用于可选的中文分词（缺失时自动退回字符二元组）。这意味着把整个目录拷到另一台电脑，装两个包就能跑起来。

数据层是一个 JSON 知识库与一个 SQLite 数据库。前者沉淀「教什么、怎么判」，可以进版本管理、可以被教师审阅；后者只存「这个学生现在什么样」，随时可以清空重来。二者分离，使得知识可以复用而学情可以重置。

## 6.2 与设计的一处偏离：为什么换掉了句向量

设计阶段的第三章把「句向量 + 余弦相似度」写成覆盖匹配的核心。实现时遇到一个现实约束：开发环境的 PyPI 官方源不可达，中文句向量模型也无法下载。于是改为「字符二元组 Dice 相似度 + 关键词组命中率」的双通道打分，再用匈牙利算法求全局最优指派。

这不是简单的降级。关键词组是人工为每条断言编写的，例如「排开 / 排出 / 溢出 / 挤出去」被定义为一组同义替换——它把教师心中「必须说到什么」的判断，直接编码成了匹配规则；而 n-gram 负责兜住口语里的语序变化与残缺表达。两者加权之后，系统在四条样例讲解上能稳定区分「讲对」「讲反」与「没讲到」，且单句匹配在毫秒级完成。

代价是清楚的：当学生用完全不同的说法表达同一个意思、又不含任何关键词时，会漏判。例如把「排开的液体」说成「被它挤走的那杯水」。接入句向量模型即可缓解，编码器在代码里是一个可替换的模块，接口已经预留。这一节如实写出，是因为工程上的取舍本身也是设计的一部分。

## 6.3 六个界面的实现

界面按第五章的设计稿还原，用原生 HTML / CSS / JavaScript 实现，没有构建步骤，由服务端静态托管。所有数字与文字都来自接口实时计算，页面里没有写死的示例数据。

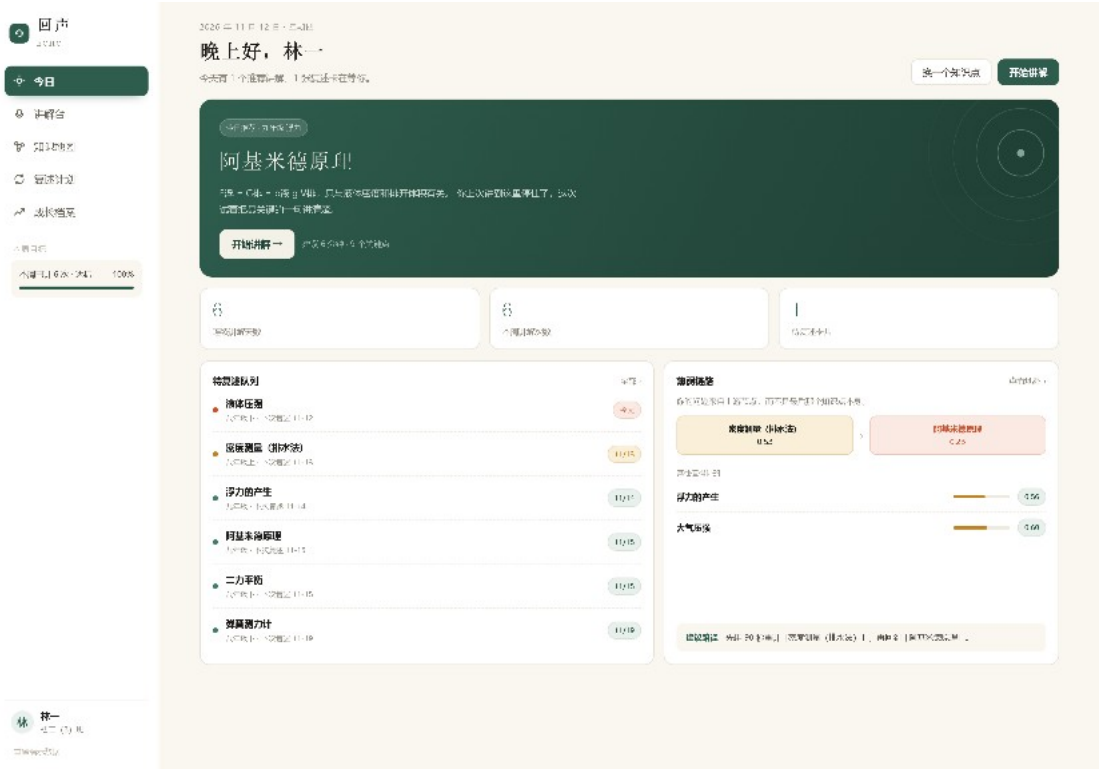



图 11　首页实现效果

首页把「今天该讲什么」做成唯一焦点，下方是待复述队列与薄弱链路。薄弱链路不再是把所有低分节点排成一列，而是从「学过但没学透」的节点往上追两层前驱，因此它显示的是一条真实的依赖路径。

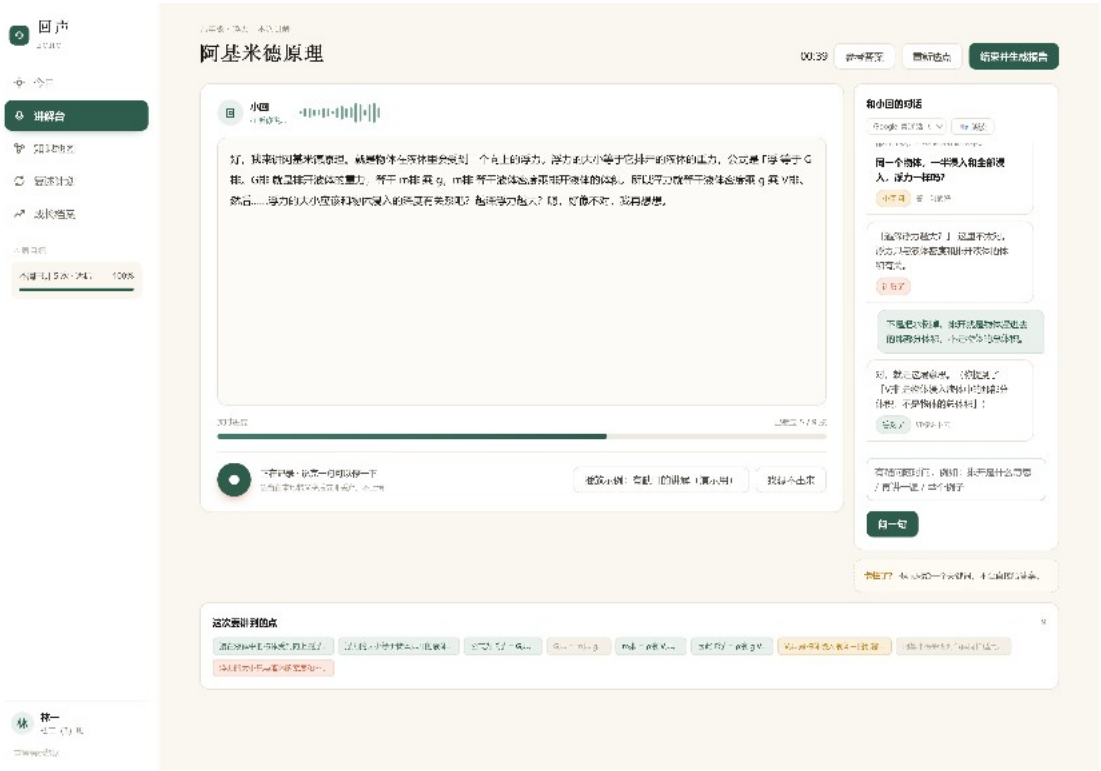



图 12　讲解台实现效果

讲解台右侧是和「小回」的对话。追问不是随口发问，而是从探针库里按信息增益挑出来的：哪条断言当前最「不确定」，就问哪一条。学生可以用语音或打字直接回答，系统会把回答重新与断言比对，给出「答对了 / 答得不全 / 还不对」的判定——答得不全时会指出漏掉的具体要素，答错时会再补一个探针。小回的问题还可以朗读出来，形成真正的一问一答。下方的断言标签随讲述实时变色：绿色讲到位，黄色一带而过，红色讲反了。

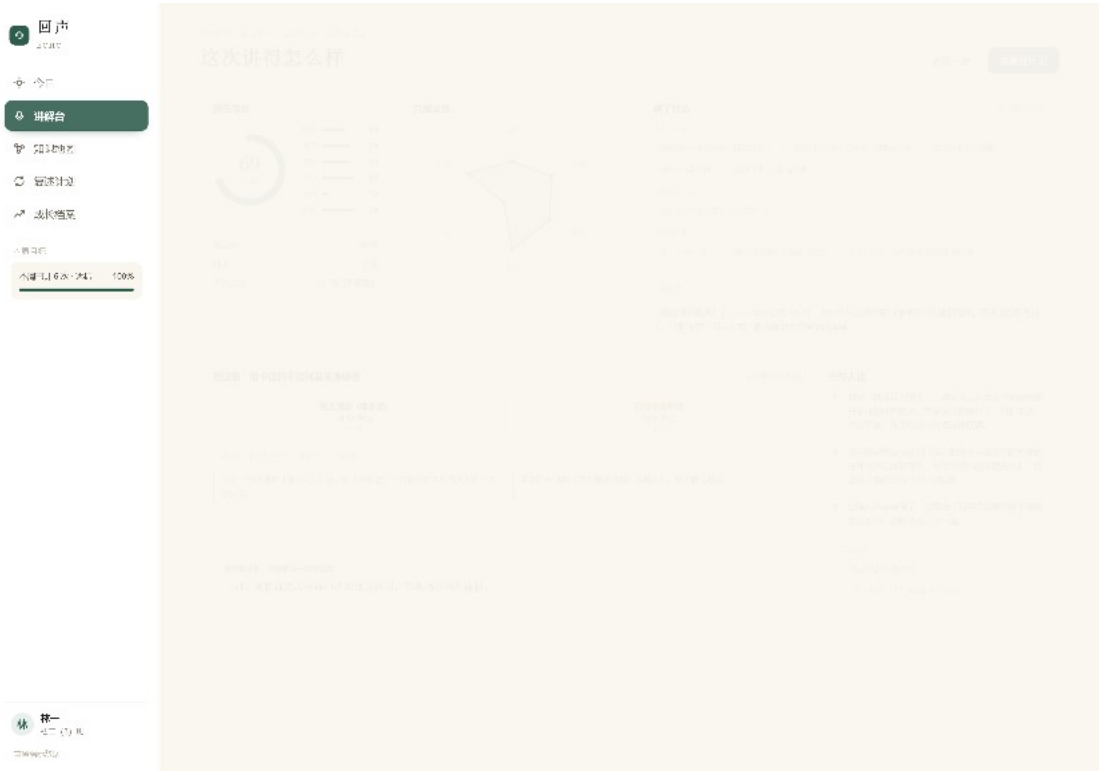



图 13　体检报告实现效果

追问的触发规则也一并收紧了。现在的策略是：只对「学生刚说过、但说得含糊」的断言提问——命中的句子必须落在最近三个匹配单元之内，问起来才像顺着话头接的；完全没提到的断言不提问，改成一句提醒，告诉他漏了哪一点；已经讲透的不问。实测下来，学生讲到 60 字左右还只会被安静地听着，直到他自己说出一个含糊的说法，小回才开口。

另外补了一页「知识点参考」（图 17）。学生随时可以翻看这个知识点的标准讲解、必须讲到的九条断言，以及三条常见错误和对应纠正。它解决的是一个很实际的问题：被追问住之后，学生需要立刻有一个地方把正确说法读一遍，而不是继续在对话里被问。这一页在讲解台里也能一键呼出——点「参考答案」会从右侧滑出同样的内容，不跳转、不打断当前会话，看完关掉继续讲。抽屉里的每一条断言都可以点——点哪一条，小回就拿哪一条来问你，相当于学生自己指定了这一轮的考点。页面上的标准讲解可以直接朗读，也能一键「按这个讲一遍」回到讲解台。

抽屉里点任意一条断言，小回就拿这一条来发问——相当于学生自己指定了这一轮的考点。图 18 是抽屉展开的样子：左边讲解照常进行，右边是随时可以翻的参考答案。




图 14　知识地图实现效果

朗读过程中也可以随时打断提问。系统把问题分成六类：问某个词是什么意思、要求把这一句展开讲、要一个例子、问原因、暂停、继续；每一类都从知识库里取现成内容——22 条术语表、标准讲解范文、断言单元、迷思纠正——答不上来就直说，并把标准说法再讲一遍，不装作懂。这样一来，朗读就不是单向播放，而是可以随时追问的讲解。

体检报告是全系统的收敛点。左侧综合分环与六维分项，中间雷达，右侧把九条断言分成三组；下半部分是根因链，它按算法返回的真实路径渲染，而不是把所有根因并排罗列。右下角的「原话摘录」把学生那句讲反的话原样摘出来。

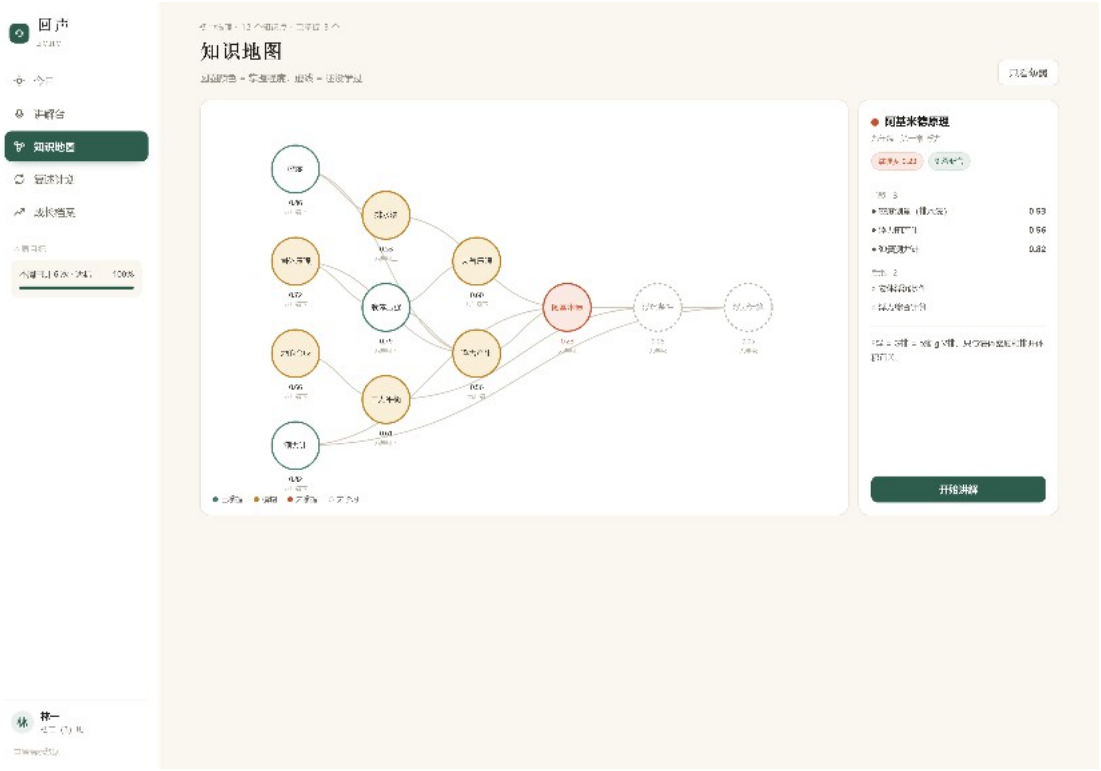



图 15　复述计划实现效果

知识地图按拓扑层级手工布局，同一章节的节点位置稳定，便于形成空间记忆。节点颜色即掌握度，虚线表示尚未学习，点击后右侧给出前驱、后继与一键跳讲。

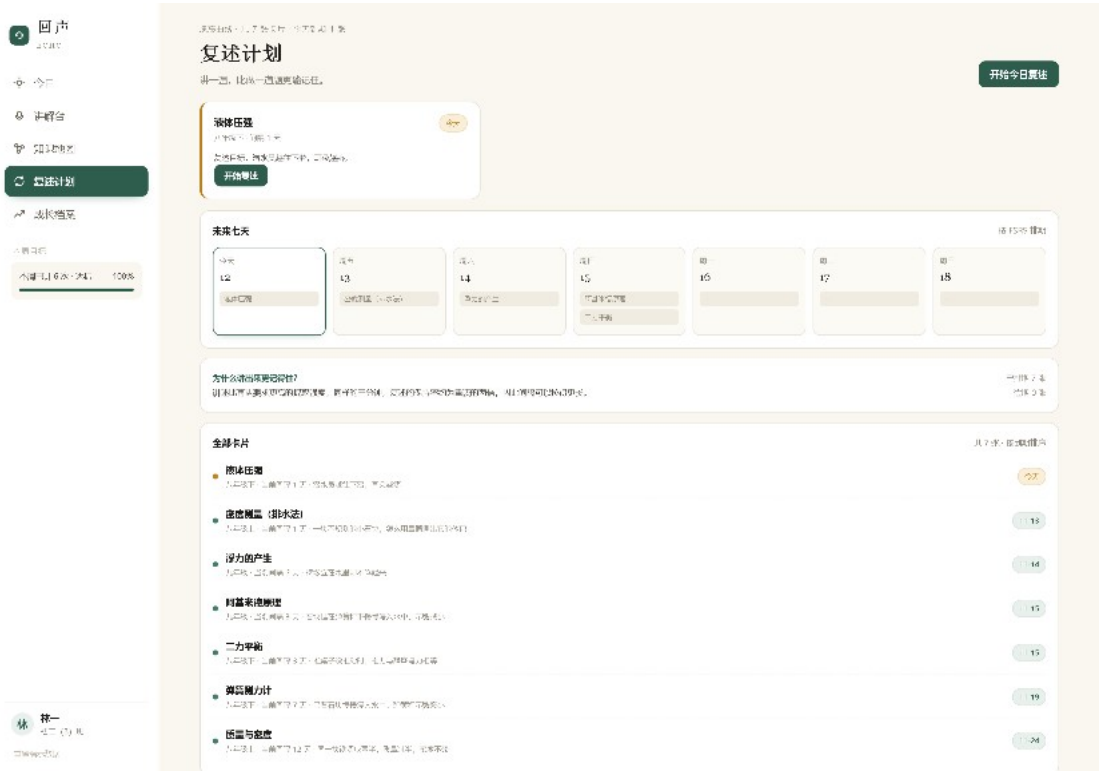



图 16　成长档案实现效果

复述计划把「今天要做什么」放在最上面，写明复述目标而不是只给知识点名字；下方是未来七天与全部卡片，每张卡片都标着当前间隔，让人看得见间隔正在变长。




图 17　知识点参考页

成长档案的折线记录的是讲解质量而非考试分数，配文是「不是分数曲线，是你讲得越来越像老师的证据」。

参考页把「教什么」摆在了明处：标准讲解给示范，必讲要点给清单，常见错误给对照。学生可以把它当成考前速览，也可以在讲完之后回来核对。




图 18　参考答案抽屉与点击提问

## 6.4 一次完整演示的数据链路

以样例「有缺口的讲解」为例，系统内部实际发生的计算是：8 句话被切成 13 个匹配单元，与 9 条断言构成 13×9 的相似度矩阵，匈牙利算法给出 5 条「讲到位」、1 条「一带而过」、3 条「没提到」，加权覆盖率 64%。同时，触发词命中了迷思 M-17，于是「与深度无关」这条断言被直接判为「讲反了」，不再计入覆盖。

六维评分得到覆盖 64、准确 78、逻辑 74、术语 87、举例 15、流畅 86，加权合成综合分 69。沿图反推两层，根因链为「密度测量（排水法）0.47 → 阿基米德原理」，因为前者的掌握度低、且在依赖路径上离目标最近。该生对阿基米德的掌握度由 0.34 更新为 0.23，并按复习等级 2 生成三天后的复述卡。

这条链路上没有一个数字是预先写死的。换一段讲解、换一个学生，结果就不同。

## 6.5 一键启动与演示路径

把系统跑起来只需要双击一次。仓库根目录的 start.bat 会做四件事：先在本地探测 8771 端口，如果服务已经在跑就直接用 Edge 打开；否则按 py -3.12、py -3.13、python 的顺序找一个能加载依赖的解释器；启动后每 0.4 秒探测一次 /api/health，直到服务就绪；最后用 Edge 打开页面。整个过程不需要命令行知识，也避免了「脚本起来了但浏览器打开时服务还没好」的常见翻车。

之所以要挑解释器，是因为依赖是编译好的 wheel（cp312），Python 版本不匹配就加载不了。启动器用一个探针语句逐个体检，选出能用的那个；都不可用时给出明确的安装命令，而不是抛一堆红字。服务在后台运行，日志写到 应用/logs，stop.bat 一键结束。

演示也做了快捷入口：网址后面加 ?demo=arch_bad 会自动播放一段有缺口的讲解并直接跳到报告页；加 &hold=1 停在讲解台，加 &answer=1 自动完成一轮问答，加 &drawer=1 把参考答案抽屉打开。答辩现场只要记三条地址，不用临场敲字。

下面这张表把全文的亮点收敛到一处——每一条都对应代码里的一处实现，以及一个可以当场复现的现象。

表 7　亮点、做法与可验证的证据

| 亮点 | 具体做法 | 可当场验证的证据 |
| --- | --- | --- |
| 让 AI 当学生 | 追问由「讲述的缺口」驱动，不是题库推送 | 讲不满 30 字不打扰；讲到含糊处才开口 |
| 评讲解，不评答案 | 61 条断言做判据，六维评分 | 同一条知识点换种讲法，分数就不同 |
| 诊断可解释 | DAG + BKT + 路径衰减，输出根因链 | 报告直指「密度测量 0.47 → 阿基米德原理」 |
| 追问可控 | 只问刚说过的、只问含糊的、答错才补问 | 每条追问都带 binds 与信息增益，可回溯 |
| 边讲边查 | 参考答案抽屉，点断言即触发提问 | 不离开讲解台，不打断会话 |
| 朗读可打断 | 六类意图的答疑，从知识库取答案 | 问「排开是什么意思」当场解释并朗读 |
| 极轻量 | 诊断引擎纯标准库，无模型依赖 | 冷启动不到 1 秒，断网也能跑 |

### 6.6 已知问题

其一，浏览器语音识别依赖在线服务，离线环境下不可用，因此演示走「文字输入」与「逐步播放示例」两条路径；语音通道保留了接口，联网即可启用。

其二，每个知识点目前只有 5 至 9 条断言，覆盖不了全部讲法，断言库仍需在真实课堂中扩充，这是当前影响覆盖率准确性的最大因素。

其三，多人并发尚未压测。班级场景下四十人同时讲述，SQLite 的写并发与单个进程的吞吐都需要验证，必要时换成 WAL 模式或轻量队列。

## 七、总结

### 7.1 创新点小结

第一，范式上的反转：把 AI 从「判官」变成「学生」。市面上的学习产品几乎都在出题、判题、推题，评价的对象是答案；回声评价的是「你把这个概念讲清楚了吗」。这

个反转不是包装，它改变了整个系统的输入——从一道题的答案，变成一段完整的、带因果的口语表达。

第二，诊断落到了「可解释」。报告不说「你没掌握浮力」，而是给出一条连通的根因链：密度测量 0.47 → 阿基米德原理。这条链由贝叶斯知识追踪和图上的路径衰减算出来，每一个数字都能回指到具体的断言、具体的原话和具体的依赖边。学生看完知道自己该回去补哪一节，教师看完知道下一节课讲什么。

第三，追问是克制的。系统不会在学生刚开口时就抛问题，也不会去问他还没讲到的概念——只在他自己提到、却说得含糊的地方开口，而且问题必须落在最近几个匹配单元里。完全没讲到的内容改成一句提醒，已经讲透的不问。这一点是实测调出来的：最初版本 59 字就会追问，看起来很像在「抽查」，改完之后才像在「听课」。

第四，把「讲—问—答—查」做成了一个闭环。被追问住的时候，学生可以随时拉出参考答案看一遍，可以点某一条断言让小回就拿它来问，也可以对着朗读直接问「排开是什么意思」。答疑不是让模型自由发挥，而是从 22 条术语表、标准讲解、断言与迷思里取现成答案，答不上来就直说，并把标准说法再讲一遍。

第五，工程上是真的轻。诊断引擎只用 Python 标准库：图遍历、匈牙利匹配、BKT、FSRS 全部手写，不依赖 NumPy、scikit-learn 或 NetworkX，也没有任何模型文件。外部依赖只有 FastAPI 与可选的 jieba。一台没有独立显卡的笔记本，双击一次就能跑起来，断网也不影响。这个取舍在本报告 6.2 节里写明了代价：换成完全不同的说法时可能漏判，接入句向量模型即可缓解。

## 7.2 局限

其一，当前设计只覆盖理科中结构清晰、断言明确的陈述性知识。对作文、历史论述这类开放性表达，断言单元的拆分会变得主观，方法的有效性需要另行验证。

其二，语音识别对方言与专业术语仍不友好，虽然提供了文字输入兜底，但这会削弱“讲出来”的自然感。

其三，评分仍依赖教师校准。系统能指出“哪里没讲到”，但对“讲得好不好”的判断，短期内无法脱离人的参照。

## 7.3 下一步

近期将完成浮力章节的完整实现与四十人试点，重点验证两件事：根因链的结论是否被教师认可，以及学生在看到报告后是否愿意再讲一遍。中期扩展到力学其余章节，并补充教师端。远期希望把“讲述”这一模态扩展到更多学科，让“讲一遍”成为和“做一套题”同等自然的学习动作。

回到最初的那句话：讲一遍，才知道自己哪里不会。这个朴素的道理人人都懂，缺的只是一个愿意听、听得懂、还会追问的听众。回声想做的，就是那个听众。